Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №3 по курсу «Операционные системы»

Группа: М8О-214БВ-25

Студент: Богатырева Е.Д.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 05.11.26

Москва, 2026

Постановка задачи

Вариант 2.

Родительский процесс создаёт дочерний процесс. Родительский и дочерний процессы представлены разными программами. Взаимодействие между процессами осуществляется через разделяемую память (POSIX Shared Memory). Пользователь вводит команды вида: «число число число<endline>». Далее эти числа передаются от родительского процесса в дочерний. Дочерний процесс считает их сумму и выводит её в файл. Числа имеют тип float. Количество чисел может быть произвольным. *Ограничение:* запрещено использовать устаревший SysV API (shmget, shmat), необходимо использовать shm_open и mmap.

Общий метод и алгоритм решения

Использованные системные вызовы и функции:

- `int shm_open(const char *name, int oflag, mode_t mode);` – создаёт (родитель, флаги O_CREAT | O_EXCL) или открывает (потомок) именованный объект разделяемой памяти POSIX.
- `int ftruncate(int fd, off_t length);` – задаёт размер объекта разделяемой памяти, равный размеру структуры shared_t.
- `void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);` – отображает объект в адресное пространство процесса (PROT_READ | PROT_WRITE, MAP_SHARED), благодаря чему оба процесса работают с одними и теми же физическими страницами.
- `int munmap(void *addr, size_t length);` – снимает отображение памяти.
- `int shm_unlink(const char *name);` – удаляет имя объекта разделяемой памяти из системы.
- `pid_t fork(void);` – создаёт дочерний процесс.
- `int execl(const char *path, const char *arg, ...);` – заменяет образ памяти дочернего процесса программой child.
- `pid_t waitpid(pid_t pid, int *status, int options);` – ожидает завершения дочернего процесса и получает код его завершения.
- `int close(int fd);` – закрывает файловый дескриптор.
- `void exit(int status);` и `void _exit(int status);` – завершают процесс с заданным статусом (_exit используется в потомке при ошибке execl).
- `pthread_mutex_*`, `pthread_cond_*` и атрибуты `pthread_mutexattr_setpshared`, `pthread_condattr_setpshared` (PTHREAD_PROCESS_SHARED) – мьютекс и условные переменные, размещённые в разделяемой памяти и доступные обоим процессам; обеспечивают взаимное исключение и ожидание событий «буфер не пуст» / «буфер не полон».
- Вспомогательные функции стандартной библиотеки: `getline` (чтение строки со стандартного ввода), `strtod` (разбор числа с плавающей точкой),

`fopen`/`fprintf`/`fflush` (запись результата в файл), `perror` (вывод описания системной ошибки).

Общая схема работы

Программа состоит из трёх файлов: общего заголовка common.h, программы родительского процесса parent.c и программы дочернего процесса child.c. Обмен данными организован по схеме «производитель – потребитель» через кольцевой буфер, лежащий в разделяемой памяти.

В common.h описана структура shared_t, которая целиком размещается в отображаемом объекте. Она содержит: мьютекс mtx; две условные переменные not_empty и not_full; индексы head (откуда читает потомок) и tail (куда пишет родитель); флаг finished_flag, означающий, что ввод закончился; массив из SLOT_COUNT = 32 ячеек slot_t. Каждая ячейка хранит число типа double (value) и признак is_eol – маркер конца строки.

Родительский процесс (parent)

1. Принимает через argv путь к исполняемому файлу дочерней программы. Создаёт объект разделяемой памяти с помощью shm_open (O_EXCL защищает от запуска поверх уже существующего объекта), задаёт размер через ftruncate и отображает его через mmap.

2. Инициализирует мьютекс и две условные переменные с атрибутом PTHREAD_PROCESS_SHARED – без него они не работали бы между разными процессами. Обнуляет head, tail и finished_flag.

3. Вызывает fork. В потомке закрывается дескриптор разделяемой памяти и вызывается execl для запуска программы child; если execl не удался, потомок завершается со статусом 127.

4. В цикле читает строки со стандартного ввода функцией getline. Каждая строка разбивается на токены функцией strtod; корректные числа помещаются в буфер функцией push с is_eol = 0. Некорректный токен пропускается с сообщением в stderr. После последнего числа строки в буфер помещается маркер конца строки (is_eol = 1).

5. Функция push захватывает мьютекс, при заполненном буфере ожидает на условной переменной not_full, записывает ячейку в позицию tail, сдвигает tail по модулю SLOT_COUNT и сигналом not_empty будит потомка. В буфере всегда остаётся одна свободная ячейка, чтобы отличать состояние «пуст» (head == tail) от «полон».

6. После окончания ввода (EOF) родитель под мьютексом устанавливает finished_flag = 1 и рассылает broadcast по обеим условным переменным. Затем ожидает потомка через waitpid, выводит его код завершения, уничтожает мьютекс и условные переменные, снимает отображение и удаляет объект через shm_unlink.

Дочерний процесс (child)

1. Открывает существующий объект разделяемой памяти через shm_open (без O_CREAT) и отображает его через mmap. Открывает файл result.txt на запись.

2. В цикле под мьютексом ждёт на условной переменной not_empty, пока буфер пуст и finished_flag не установлен. Если буфер пуст и ввод завершён – выходит из цикла. Иначе извлекает ячейку из позиции head, сдвигает head и сигналом not_full уведомляет родителя о свободном месте. Мьютекс удерживается только на время копирования ячейки, а вся обработка выполняется уже вне критической секции.

3. Если ячейка содержит число – прибавляет его к накопленной сумме. Если ячейка является маркером конца строки – записывает сумму в result.txt (формат %.10g) и сбрасывает накопитель. Для строки без чисел записывается «(empty line)».

4. После завершения цикла закрывает файл, снимает отображение и закрывает дескриптор. Удаление объекта разделяемой памяти выполняет только родитель.

Обработка ошибок

Все системные вызовы shm_open, ftruncate, mmap, fork, waitpid, а также fopen в потомке, проверяются на ошибку. При сбое выводится описание через perror и процесс завершается с кодом EXIT_FAILURE. Проверяется число аргументов командной строки родителя. Ошибка запуска дочерней программы (execl) приводит к завершению потомка с кодом 127, который родитель выводит на экран.

Код программы

common.h

```
#ifndef COMMON_H
#define COMMON_H

#include <pthread.h>

#define SHM_NAME    "/lab3_var2_shm"
#define SLOT_COUNT  32

typedef struct {
    double value;
    int    is_eol;
} slot_t;

typedef struct {
    pthread_mutex_t mtx;
    pthread_cond_t  not_empty;
    pthread_cond_t  not_full;

    int head, tail;
    int finished_flag;
    slot_t slots[SLOT_COUNT];
} shared_t;

#endif
```

parent.c

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <sys/wait.h>
#include "common.h"

static shared_t *shm = NULL;
static int shm_fd = -1;

static void die(const char *msg) {
    perror(msg);
    exit(EXIT_FAILURE);
}

static void cleanup(void) {
    if (shm) { munmap(shm, sizeof(shared_t)); shm = NULL; }
    if (shm_fd >= 0) {
        close(shm_fd);
        shm_unlink(SHM_NAME);
        shm_fd = -1;
    }
}

static void push(double v, int is_eol) {
    pthread_mutex_lock(&shm->mtx);
    while ((shm->tail + 1) % SLOT_COUNT == shm->head) {
        pthread_cond_wait(&shm->not_full, &shm->mtx);
    }
```

```
    shm->slots[shm->tail].value  = v;
    shm->slots[shm->tail].is_eol = is_eol;
    shm->tail = (shm->tail + 1) % SLOT_COUNT;
    pthread_cond_signal(&shm->not_empty);
    pthread_mutex_unlock(&shm->mtx);
}

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "Usage: %s <child_binary>\n", argv[0]);
        return EXIT_FAILURE;
    }

    shm_fd = shm_open(SHM_NAME, O_CREAT | O_RDWR | O_EXCL, 0600);
    if (shm_fd < 0) die("shm_open");
    if (ftruncate(shm_fd, sizeof(shared_t)) < 0) die("ftruncate");

    shm = mmap(NULL, sizeof(shared_t),
               PROT_READ | PROT_WRITE, MAP_SHARED, shm_fd, 0);
    if (shm == MAP_FAILED) die("mmap");

    pthread_mutexattr_t ma;
    pthread_condattr_t  ca;
    pthread_mutexattr_init(&ma);
    pthread_condattr_init(&ca);
    pthread_mutexattr_setpshared(&ma, PTHREAD_PROCESS_SHARED);
    pthread_condattr_setpshared(&ca, PTHREAD_PROCESS_SHARED);
    pthread_mutex_init(&shm->mtx, &ma);
    pthread_cond_init(&shm->not_empty, &ca);
    pthread_cond_init(&shm->not_full,  &ca);
    pthread_mutexattr_destroy(&ma);
    pthread_condattr_destroy(&ca);

    shm->head = shm->tail = 0;
    shm->finished_flag = 0;

    pid_t pid = fork();
    if (pid < 0) die("fork");
    if (pid == 0) {
        close(shm_fd);
        execl(argv[1], argv[1], (char *)NULL);
        perror("execl");
        _exit(127);
    }

    char *line = NULL;
    size_t cap = 0;
    ssize_t len;

    while ((len = getline(&line, &cap, stdin)) > 0) {
        char *p = line;
        while (*p) {
            while (*p == ' ' || *p == '\t' || *p == '\r') p++;
            if (*p == '\0' || *p == '\n') break;

            char *end = NULL;
            errno = 0;
            double v = strtod(p, &end);
            if (end == p) {
                fprintf(stderr, "[parent] bad token, skipping\n");
                while (*p && *p != ' ' && *p != '\t' && *p != '\n') p++;
                continue;
            }
            push(v, 0);
            p = end;
        }
```

```
            push(0.0, 1);
        }
        free(line);

        pthread_mutex_lock(&shm->mtx);
        shm->finished_flag = 1;
        pthread_cond_broadcast(&shm->not_empty);
        pthread_cond_broadcast(&shm->not_full);
        pthread_mutex_unlock(&shm->mtx);

        int status = 0;
        if (waitpid(pid, &status, 0) < 0) die("waitpid");

        if (WIFEXITED(status))
            printf("[parent] child exited with code %d\n", WEXITSTATUS(status));
        else if (WIFSIGNALED(status))
            printf("[parent] child killed by signal %d\n", WTERMSIG(status));

        pthread_mutex_destroy(&shm->mtx);
        pthread_cond_destroy(&shm->not_empty);
        pthread_cond_destroy(&shm->not_full);

        cleanup();
        return EXIT_SUCCESS;
    }
```

child.c

```
    #define _GNU_SOURCE
    #include <stdio.h>
    #include <stdlib.h>
    #include <unistd.h>
    #include <fcntl.h>
    #include <sys/mman.h>
    #include <sys/stat.h>
    #include "common.h"

    static shared_t *shm = NULL;
    static int shm_fd = -1;
    static FILE *out_fp = NULL;

    static void die(const char *msg) {
        perror(msg);
        exit(EXIT_FAILURE);
    }

    static void cleanup(void) {
        if (out_fp) { fclose(out_fp); out_fp = NULL; }
        if (shm) { munmap(shm, sizeof(shared_t)); shm = NULL; }
        if (shm_fd >= 0) { close(shm_fd); shm_fd = -1; }
    }

    int main(void) {
        shm_fd = shm_open(SHM_NAME, O_RDWR, 0);
        if (shm_fd < 0) die("shm_open (child)");

        shm = mmap(NULL, sizeof(shared_t),
                   PROT_READ | PROT_WRITE, MAP_SHARED, shm_fd, 0);
        if (shm == MAP_FAILED) die("mmap (child)");

        out_fp = fopen("result.txt", "w");
        if (!out_fp) die("fopen result.txt");

        double sum = 0.0;
```

```
    int have_numbers = 0;

    for (;;) {
        pthread_mutex_lock(&shm->mtx);

        while (shm->head == shm->tail && !shm->finished_flag) {
            pthread_cond_wait(&shm->not_empty, &shm->mtx);
        }
        if (shm->head == shm->tail && shm->finished_flag) {
            pthread_mutex_unlock(&shm->mtx);
            break;
        }

        slot_t s = shm->slots[shm->head];
        shm->head = (shm->head + 1) % SLOT_COUNT;
        pthread_cond_signal(&shm->not_full);
        pthread_mutex_unlock(&shm->mtx);

        if (s.is_eol) {
            if (have_numbers)
                fprintf(out_fp, "%.10g\n", sum);
            else
                fprintf(out_fp, "(empty line)\n");
            fflush(out_fp);
            sum = 0.0;
            have_numbers = 0;
        } else {
            sum += s.value;
            have_numbers = 1;
        }
    }

    cleanup();
    return EXIT_SUCCESS;
}
```

Сборка

```
gcc -Wall -Wextra -o parent parent.c -pthread -lrt
gcc -Wall -Wextra -o child child.c -pthread -lrt
```

Протокол работы программы

Тест 1: обычные строки, пустая строка, некорректный токен

```
$ cat in.txt
1.5 2.5 3
10 20 30.25

-4.5 4.5
abc 5 6
7
$ ./parent ./child < in.txt
[parent] bad token, skipping
[parent] child exited with code 0
$ cat result.txt
7
60.25
(empty line)
0
11
7
```

Суммы совпадают с ожидаемыми: 1.5 + 2.5 + 3 = 7; 10 + 20 + 30.25 = 60.25; пустая строка обработана отдельно; -4.5 + 4.5 = 0; токен «abc» пропущен, 5 + 6 = 11; 7.

Тест 2: 200 строк, в строке с номером i записано i единиц

```
$ ./parent ./child < big.txt
[parent] child exited with code 0
$ wc -l result.txt
200 result.txt
$ head -3 result.txt
1
2
3
$ tail -2 result.txt
199
200
```

Суммарно через буфер из 32 ячеек пройдено 20 100 чисел, то есть буфер заполнялся многократно; синхронизация через мьютекс и условные переменные отработала без потерь и взаимных блокировок.

Тест 3: пустой ввод

```
$ ./parent ./child < /dev/null
[parent] child exited with code 0
$ wc -c result.txt
0 result.txt
```

Тест 4: ошибки

```
$ ./parent
Usage: ./parent <child_binary>
$ ./parent ./nonexist < in.txt
execl: No such file or directory
[parent] child exited with code 127
```

Вывод

В ходе лабораторной работы была реализована программа из двух процессов, обменивающихся данными через отображаемую разделяемую память (shm_open + mmap) вместо каналов. Для корректной работы кольцевого буфера использованы мьютекс и условные переменные с атрибутом PTHREAD_PROCESS_SHARED, размещённые непосредственно в разделяемой памяти. Программа правильно вычисляет суммы вещественных чисел для строк произвольной длины, обрабатывает пустые строки и некорректные токены, а также проверяет коды возврата системных вызовов. Основная сложность заключалась в организации синхронизации между разными процессами: необходимо было корректно инициализировать примитивы с флагом общего доступа, не потерять сигнал о завершении ввода (для этого используются флаг finished_flag и broadcast) и освободить разделяемый объект через shm_unlink.